

PRÀCTICA 1 - MongoDB

Arquitectura y Tecnologías del Software

Nil Camprubí Vilademunt (1671714)
Lluc Bertran Canicio (1671450)

TASQUES OBLIGATORIES

1. Diseny de l'esquema de la base de dades

Després d'analitzar la estructura de dades dels datasets, hem vist que el tipus de relació entre aquests depèn del cas d'ús que se li apliqui. Per això hem pensat un cas d'ús a partir dels quals treballarem els problemes d'aquesta pràctica:

Cas d'ús

Se'ns ha encarregat dissenyar la base de dades NoSQL per a una aplicació de recomanació de restaurants, similar a TripAdvisor. L'aspecte en que es diferencia és la gran importància que l'aplicació dona a les inspeccions sanitàries a l'hora de recomanar restaurants. És per això que aquesta aplicació ha de gestionar informació sobre restaurants i les seves inspeccions sanitàries per oferir recomanacions als usuaris.

Les dades necessàries són les següents:

- Restaurants: nom, tipus de cuina, adreça, valoració i url del lloc web del restaurant.
- Inspeccions sanitàries: data de l'última inspecció, resultat i sector d'aquesta.

D'inici, disposem de dos datasets:

1. Un amb tots els restaurants registrats a l'aplicació.
2. Un altre amb totes les inspeccions sanitàries realitzades.

Hem de dissenyar una estructura i esquema de base de dades que permeti optimitzar les consultes, ja que se'n faran moltes, tantes com cerques de restaurants facin els usuaris. Això inclou:

- Consultes ràpides per nom, tipus de cuina o ubicació.
- Recuperació eficient de la informació d'inspeccions d'un restaurant.

A més, caldrà tenir en compte com es gestionaran les actualitzacions, com l'entrada de nous restaurants i l'afegit de noves inspeccions, per garantir que el sistema sigui escalable i eficient.

Un cop definit el cas d'ús i les consultes principals que es faran, passem a fer el disseny de l'esquema de la base de dades.

Hem decidit fer un tipus de relació **One-to-Many embedded** entre el dataset **restaurants** i **inspeccions**, ja que desde la aplicació descrita en el cas d'ús, interessa poder accedir a la informació detallada de tots els restaurants, i a més a més, poder tenir informació bàsica de les inspeccions que a mesura que el temps passi i l'app prosperi, podrien ser moltes, d'aquí a escollir "one-to-many" (actualment ja n'hi ha un que en té 17).

Per tant hem creat una nova colecció a partir de la de 'restaurants' afegint els camps que ens interessin de les inspeccions (id, data i el resultat).

Hem decidit no agafar més camps (com "sector" o "certificate_number") perquè no se'n faràn tantes consultes. Ja que tenim pensat que la consulta principal de la app mostra la informació dels restaurants i la data i resultat de les inspeccions. Si l'usuari volgues més informació ja es faria una consulta al propi dataset de inspeccions.

Codi per la creació de 'restaurant-inspeccio':

```
db.restaurants.aggregate([
  {
    $addFields: {
      idAsString: { $toString: "$_id" }
    }
  },
  {
    $lookup: {
      from: "inspections",
      localField: "idAsString",
      foreignField: "restaurant_id",
      as: "inspections"
    }
  },
  {
    $project: {
      _id: 0,
      name: 1,
      address: {
        street: "$address",
        city: "$address line 2",
        outcode: "$outcode",
        postcode: "$postcode"
      },
      rating: 1,
      type_of_food: 1,
      url: 1,
      inspections: {
        $map: {
          input: "$inspections",
          as: "inspection",
          in: {
            date: "$$inspection.date",
            result: "$$inspection.result",
            sector: "$$inspection.sector"
          }
        }
      }
    }
  },
  { $out: "restaurant-inspections" }
]);
```

Aquest codi ens crea aquesta nova colecció “restaurant-inspections” la qual té les inpeccions en un array dins el document del restaurant.

```
  _id: ObjectId('67da8a31540fe2af2c0d25ca')
  ▶ address: Object
    name: ".CN Chinese"
    rating: 5
    type_of_food: "Chinese"
  ▼ inspections: Array (3)
    ▼ 0: Object
      date: "Dec 26 2023"
      result: "Fail"
      sector: "Cigarette Retail Dealer - 127"
    ▼ 1: Object
      date: "Feb 20 2025"
      result: "Fail"
```

Codi per l'**esquema de validació** (jsonScheme):

```
db.runCommand({
  collMod: "restaurant-inspections",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [ "name", "address", "rating", "type_of_food", "url", "inspections" ],
      properties: {
        name: { bsonType: "string" },
        address: {
          bsonType: "object",
          required: [ "street", "city", "outcode", "postcode" ],
          properties: {
            street: { bsonType: "string" },
            city: { bsonType: "string" },
            outcode: { bsonType: "string" },
            postcode: { bsonType: "string" }
          }
        },
        rating: { bsonType: "int", minimum: 1, maximum: 5 },
        type_of_food: { bsonType: "string" },
        url: { bsonType: "string" },
        inspections: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: [ "date", "result" ],
            properties: {
              date: { bsonType: "date" },
              result: {
                bsonType: "string",
                enum: [ "Fail", "Pass", "Warning Issued", "Violation Issued", "No
Violation Issued" ]
              }
            }
          }
        },
        sector: { bsonType: "string" }
      }
    }
  }
})
```



```

    $group: {
      _id: "$result",
      count: { $sum: 1 }
    }
  },
  {
    $project: {
      result: "$_id",
      count: 1,
      percentage: {
        $multiply: [
          { $divide: ["$count", totalInspections] },
          100
        ]
      },
      _id: 0
    }
  },
  {
    $sort: { percentage: -1 }
  }
])

```

Unir restaurantes con sus inspecciones utilizando \$lookup.

```

db.restaurants.aggregate([
  {
    $addFields: {
      idAsString: { $toString: "$_id" }
    }
  },
  {
    $lookup: {
      from: "inspections",
      localField: "idAsString",
      foreignField: "restaurant_id",
      as: "inspections"
    }
  }
])

```

TASQUES AVANÇADES

Aquest conjunt de tasques les farem en funció de l'esquema presentat en l'exercici 1 de la pràctica.

1. Optimizació del rendiment

Identificació de les possibles consultes més freqüents en el nostre cas d'ús.

C-01 → Restaurants amb més inspeccions sanitàries superades.

```
db.restaurants_with_inspections.aggregate([
  {
    $match: {
      "inspections.result": "Pass"
    }
  },
  {
    $unwind: "$inspections"
  },
  {
    $match: {
      "inspections.result": "Pass"
    }
  },
  {
    $group: {
      _id: "$name",
      inspeccionsSuperades: { $sum: 1 }
    }
  },
  {
    $sort: { inspeccionsSuperades: -1 }
  }
]).explain("executionStats")
```

C-02 → Restaurants que han superat l'última inspecció sanitària (inspection.result == "Pass").

```
db.restaurants_with_inspections.aggregate([
  {
    $addFields: {
      lastInspection: {
        $arrayElemAt: [
          { $sortBy: { input: "$inspections", sortBy: { date: -1 } } },
          0
        ]
      }
    }
  }
])
```

```

    ]
  }
}
},
{
  $match: {
    "lastInspection.result": "Pass"
  }
},
{
  $project: {
    name: 1,
    _id: 0
  }
}
]).explain("executionStats")

```

C-03 → Restaurants més ben valorats d'una ciutat concreta (sort rating -1) i amb l'última inspecció sanitària superada.

```

db.restaurants_with_inspections.aggregate([
  { $addFields: { latestInspection: { $arrayElemAt: [
    { $sortBy: { input: "$inspections", sortBy: { date: -1 } } }, 0 ] }
  } },
  { $match: { "latestInspection.result": "Pass", "address.city": "London" }
},
  { $sort: { rating: -1 } }
]).explain("executionStats")

```

C-04 → Restaurants més ben valorats d'un tipus de cuina concreta (sort rating -1) i amb l'última inspecció sanitària superada.

```

db.restaurants_with_inspections.aggregate([
  { $addFields: { latestInspection: { $arrayElemAt: [
    { $sortBy: { input: "$inspections", sortBy: { date: -1 } } }, 0 ] }
  } },
  { $match: { "latestInspection.result": "Pass", "type_of_food": "Chinese" }
},
  { $sort: { rating: -1 } }
]).explain("executionStats")

```

Implementar índex adequats per aquestes consultes.

Índex implementats:

```
db.getCollection("restaurant-inspections").createIndex({ "inspections.result": 1 })
db.getCollection("restaurant-inspections").createIndex({ "name": 1, "inspections.result": 1 })
db.restaurants.createIndex({ "inspections.date": -1, "inspections.result": 1 });
db.restaurants_with_inspections.createIndex({ "address.city": 1, "latestInspection.result": 1 });
db.restaurants_with_inspections.createIndex({ "rating": 1 });
db.restaurants_with_inspections.createIndex({ "type_of_food": 1, "latestInspection.result": 1 });
```

Comparar el rendiment abans i després de crear els índex utilitzant explain().

Anàlisi de rendiment amb 'explain()':

C-01:

Documents Returned: 842	Actual Query Execution Time (ms): 10 ms
Index Keys Examined: 0	Sorted in Memory: false
Documents Examined: 2548	No index available for this query. Create Index
STAGE: PROJECTION_SIMPLE Details	
nReturned: 2548	Execution Time: 0 ms
↑	
STAGE: COLLSCAN Details	
nReturned: 2548	Execution Time: 0 ms
Documents Examined 2548	

Apliquem índex:

```
db.getCollection("restaurant-inspections").createIndex({ "inspections.result": 1 })
db.getCollection("restaurant-inspections").createIndex({ "name": 1, "inspections.result": 1 })
```

A continuació hem tornat a executar la consulta i hem vist una millora de 3 ms, és a dir hem passat de 10 ms a 7 ms d'execució.

```
executionStats: {
  executionSuccess: true,
  nReturned: 1069,
  executionTimeMillis: 7,
  totalKeysExamined: 1069,
  totalDocsExamined: 1069,
```


C-02:

Documents Returned: 514	Actual Query Execution Time (ms): 33 ms
Index Keys Examined: 0	Sorted in Memory: false
Documents Examined: 2548	No index available for this query. Create Index

STAGE: PROJECTION_SIMPLE	Details
nReturned: 2548	Execution Time: 0 ms

↑

STAGE: COLLSCAN	Details
nReturned: 2548	Execution Time: 0 ms
Documents Examined: 2548	

Apliquem index:

```
db.restaurants_with_inspections.createIndex({ "inspections.date": 1 });
```

A continuació hem tornat a executar la consulta i hem vist una millora de 22 ms, és a dir hem passat de 33 ms a 11 ms d'execució, lo qual és una millora molt notable.

```
executionStats: {
  executionSuccess: true,
  nReturned: 2548,
  executionTimeMillis: 11,
  totalKeysExamined: 0,
  totalDocsExamined: 2548,
```

C-03

```
executionStats: {
  executionSuccess: true,
  nReturned: 345,
  executionTimeMillis: 3,
  totalKeysExamined: 0,
  totalDocsExamined: 2548,
```

Apliquem index:

```
db.restaurants_with_inspections.createIndex({ "address.city": 1,
"latestInspection.result": 1 });
db.restaurants_with_inspections.createIndex({ "rating": 1 });
```

Després de la creació dels índexs hem tornat a executar la consulta i hem vist una millora d'1 ms, és a dir hem passat de 3 ms, que ja era una molt bona marca, a 1 ms d'execució.

```
executionStats: {
  executionSuccess: true,
  nReturned: 345,
  executionTimeMillis: 2,
  totalKeysExamined: 345,
  totalDocsExamined: 345,
```

C-04

```
executionStats: {
  executionSuccess: true,
  nReturned: 174,
  executionTimeMillis: 5,
  totalKeysExamined: 0,
  totalDocsExamined: 2548,
```

Apliquem index:

```
db.restaurants_with_inspections.createIndex({ "type_of_food": 1,
"latestInspection.result": 1 });
db.restaurants_with_inspections.createIndex({ "rating": 1 });
```

Després de la creació dels índexs hem tornat a executar la consulta i hem vist una millora de 4 ms, és a dir hem passat de 5 ms, que era una molt bona marca, a 1 ms d'execució. És una gran millora considerant com de bo era el temps previ a l'optimització.

```
executionStats: {
  executionSuccess: true,
  nReturned: 174,
  executionTimeMillis: 1,
  totalKeysExamined: 174,
  totalDocsExamined: 174,
```

2. Estratègies de escalabilitat

Estrategia de sharding adequada per aquest dataset.

Les estratègies de sharding consisteixen en distribuir les dades de forma eficient per millorar el rendiment i garantir l'escalabilitat.

L'objectiu en el cas del nostre dataset modificat (el que té les inspeccions dins de cada restaurant), seria distribuir els restaurants de manera uniforme per minimitzar l'impacte de creixements futurs i equilibrar la càrrega entre nodes.

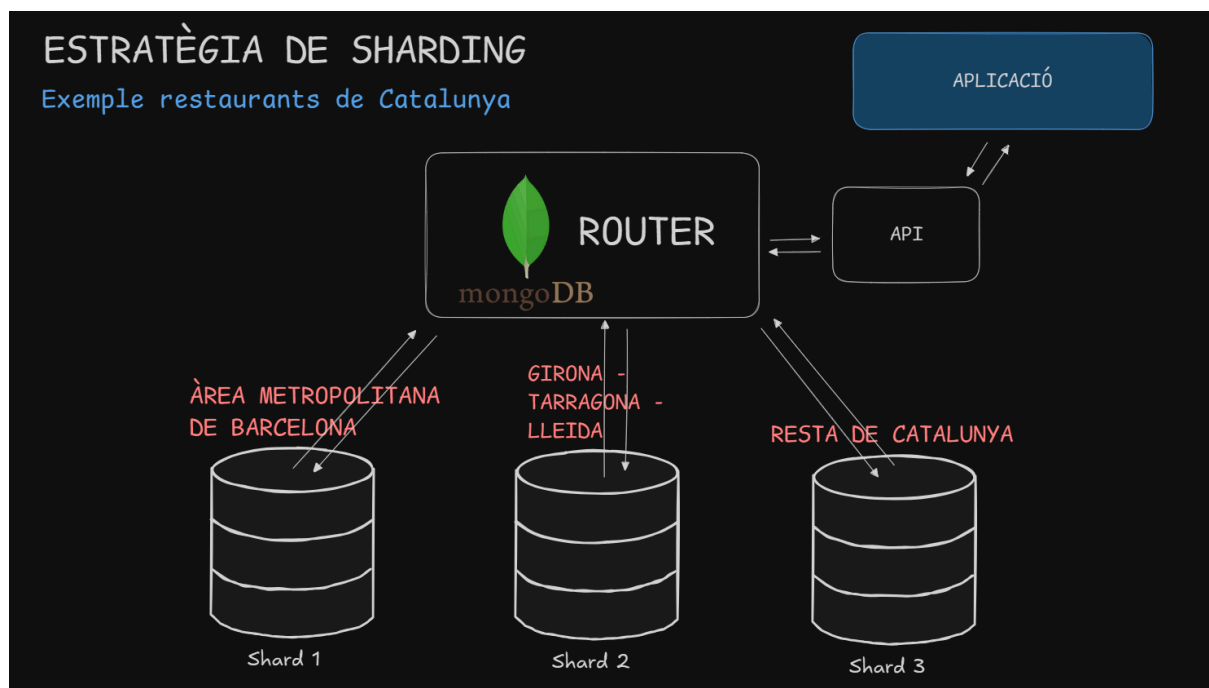
Una bona opció seria fer sharding per 'type_of_food', ja que la càrrega estaria bastant distribuïda, i ajudaria en el rendiment de les consultes relacionades amb el tipus de menjar, que són bastant comunes en el nostre cas d'ús.

Una altra opció seria fer sharding per 'city', en el nostre context la gran majoria de consultes són sobre els restaurants que l'usuari té aprop. Per tant realment milloraria molt el rendiment fent les consultes més eficients. Però no tot són punts positius, aquesta

estratègia de sharding podria requerir de masses shards, i els shards de les ciutats grans podrien estar sobrecarregats en comparació a altres.

Per tant la estratègia que creiem idònea seria fer sharding per territori, algun shard podria ser una gran ciutat, i altres podrien ser un gran conjunt de pobles i ciutats més petites. D'aquesta manera la càrrega estaria ben dimensionada i a més a més les cerques per ubicació serien eficients. A més si una ciutat creixés molt es podria moure a un nou shard sense problema i d'aquesta manera augmentar-ne l'eficiència de les consultes en aquella ubicació.

A continuació veiem un esquema de com seria la estratègia si els restaurants només fossin restaurants de Catalunya:



Diseny d'un esquema de replicació per alta disponibilitat

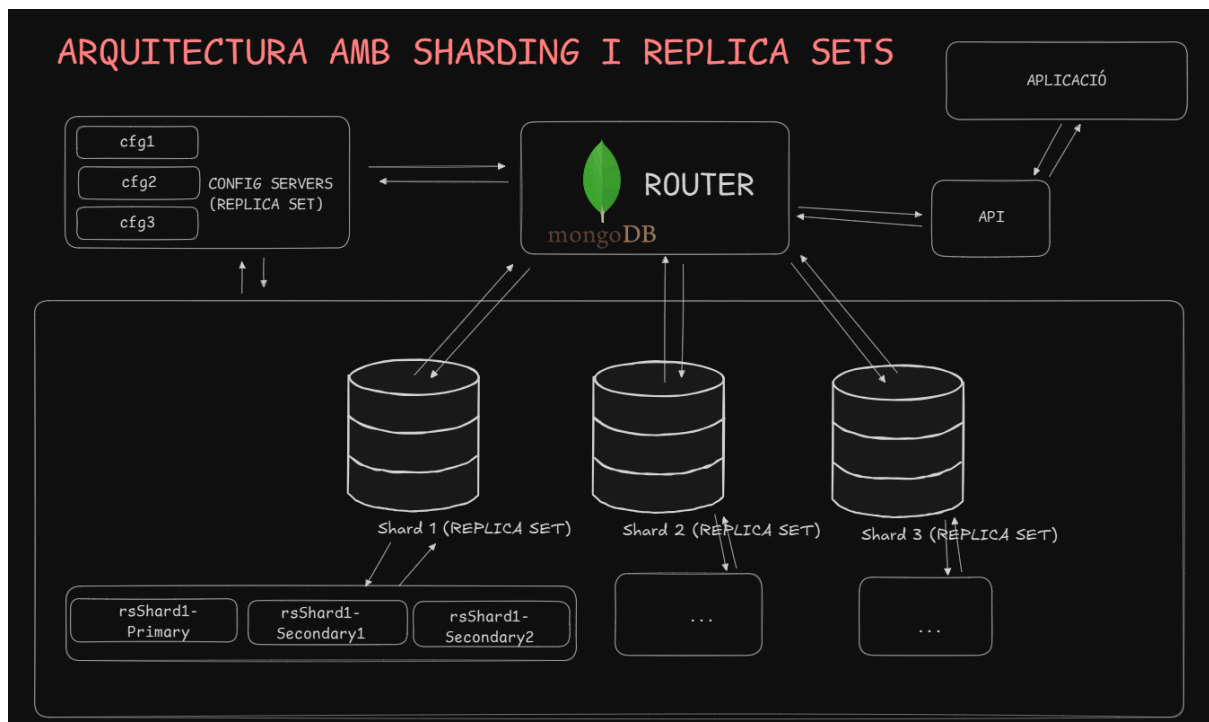
Per garantir alta disponibilitat en la base de dades, implementariem **replica sets** en cadascun dels *shards*. Això assegura que, si un node cau, la base de dades continua funcionant sense interrupcions.

Estratègia de replicació:

- Cada shard tindrà un Replica Set format per 3 nodes:
 - Primary → Accepta escriptures i lectures.
 - Secondary1 → Replica les dades del primary i pot acceptar lectures si s'activa readPreference.

- Secondary2 → Una altra rèplica del *Primary* que també replica les dades i pot acceptar lectures si s'activa *readPreference*. Pot servir per equilibrar la càrrega de lectura.
- Config Servers en Replica Set
 - Gestionen la informació sobre quines dades hi ha en cada shard.
 - També han d'estar replicats per evitar pèrdues de metadades.

Arquitectura completa amb els replica set i sharding:



Analisi de possibles *bottlenecks* i solucions a aquests.

En la arquitectura anterior amb Sharding i replica sets, poden haver diferents bottlenecks.

El principal podria ser el Router (mongos), un sol router podria tenir massa càrrega a dirigir totes les peticions als Shards adequats. La solució seria tenir múltiples instàncies de mongos per distribuir la càrrega.

Un altre coll de botella si els shards escalesin podria ser els 'config servers', ja que en l'arquitectura actual només n'hi ha un (replicat 2 vegades). Aquests servidors contenen metadades importants, i si són lents o estan massa sobrecarregats podrien alentir la gestió de dades.

Tot i això en principi l'arquitectura permet escalar tant horitzontalment com verticalment, pel que devan un bottleneck es podria aplicar una ràpida solució.